

PayliteNG

Initial Security Assessment & Incident Findings

Customer Login Portal — Authentication Infrastructure Review

Client	PayliteNG — Aba, Abia State, Nigeria
Engagement contact	Mr. Emeka Okafor, IT Manager
Report recipient	Mrs. Adaeze Obi, Chief Executive Officer
Prepared by	Onwubiko Godswill Chinaecherem, Junior Security Consultant
Date submitted	20 June 2026
Document classification	CONFIDENTIAL — Internal & Board Use Only
Reference	PNG-SEC-2026-001

This report is intended solely for PayliteNG leadership and its designated legal and compliance advisors. Distribution outside this group requires written consent from the author.

1. Executive Summary

Prepared for Mrs. Adaeze Obi, Chief Executive Officer

On the morning of 15 February, an attacker reached PayliteNG's server through our SSH remote-access service and logged in using the root account — the account with unrestricted control over the system. Once inside, the intruder immediately opened a command shell, the digital equivalent of getting the keys to every door in the building at once.

This did not happen because the attacker was clever. It happened because the front door was left wide open. Our logs show the attacker simply guessed the root password correctly on the tenth attempt, in under ten seconds, with no alarm raised and no lockout triggered. Three other accounts on the same server were probed the same morning, and two of those were also breached using the same guess-repeatedly approach.

We have no evidence in the available logs that customer financial data was copied or altered, but we also do not yet have visibility into what happened after the attacker gained root access, since file-level and database logging was not active at the time. Given that this is a fintech platform handling customer funds, we are treating this as a reportable incident pending further forensic review.

Severity

CRITICAL

An external actor obtained full administrative (root) access to a production-adjacent server in under six minutes of effort, using a basic password-guessing attack that should have been blocked by standard controls.

Top 3 actions PayliteNG must take in the next 24 hours

1. Lock down remote root access. Disable the ability to log in as root over SSH and force every administrator onto named, individually traceable accounts with multi-factor authentication. This single change would have stopped this incident before it started.
2. Throttle and lock out repeated login failures. Install a brute-force protection tool (such as Fail2Ban) so that ten wrong passwords in ten seconds results in an automatic IP ban, not a tenth chance.
3. Freeze and audit the affected server. Treat the box touched by 203.0.113.77 as compromised until proven otherwise — rotate every credential that was ever used on it, and bring in forensic logging before doing anything else on that machine.

The remainder of this report sets out exactly what we found, how we found it, and a full set of recommendations Mr. Okafor's team can begin implementing immediately. None of this requires new hardware or significant budget, it requires configuration changes that should already have been in place.

2. Asset Inventory

The table below lists the PayliteNG systems, services, and data assets identified during Week 1 and Week 2 reconnaissance activity, scoped to the authentication and access layer that is the subject of this assessment. Assets are grouped by where they sit in the environment and tagged with a rough sensitivity rating, since that drives how everything is prioritized later in this report.

Asset	Type	Description	Sensitivity
Auth/Login Server (target host)	Server ; Linux	Handles SSH-based administrative access for the PayliteNG server replica; the host directly implicated in this incident.	Critical
Customer Login Portal	Web application	Customer-facing authentication front end for the PayliteNG fintech platform; sits behind the same network perimeter as the affected host.	Critical
sshd service (port 22)	Network service	Remote administrative access daemon; the entry point used in this incident by all four IPs observed.	High
Local user accounts (admin, ubuntu, root, guest, alice, bob, charlie)	Identity / access	Operating-system level accounts with login or sudo rights; several were targeted and compromised.	High
sudoers configuration	Access control config	Governs which local accounts can elevate privileges; observed being used to spawn a root shell.	High
System authentication logs (/var/log/auth.log equivalent)	Log data	Primary evidentiary source for this report; the only forensic artefact currently retained for this host.	Medium
Apache web service	Network service	Restarted mid-incident by an internal account (bob); likely fronts the customer portal.	Medium
Customer account & transaction data (presumed downstream)	Data asset	Not directly observed in this log excerpt, but is the asset PayliteNG ultimately needs protected behind this authentication layer.	Critical

Note: this inventory reflects what is observable from the authentication log and from the engagement scope. A full asset register covering the production database, payment-processing integrations, and CBN-regulated transaction stores was out of scope for this initial assessment and is recommended as a follow-on engagement.

3. Threat Actor Profile

Who is @gr4yh4t?

@gr4yh4t is a handle that surfaced during our open-source intelligence (OSINT) sweep of forums and pastebin-style sites where compromised credentials and target lists for Nigerian and West African fintech infrastructure are occasionally traded. The handle does not correspond to a known, named threat group tracked by major vendors — it reads as an individual or small crew operating at the low-to-mid end of the skill spectrum, using publicly available tooling rather than custom malware.

The naming convention itself is a small tell: “grey hat” framing is common among operators who want to project a hacktivist or researcher identity even when their actual conduct — unauthorised access, no prior disclosure, no engagement with the victim — is indistinguishable from criminal intrusion. We treat the self-description as marketing, not as a mitigating fact.

What is known from infrastructure and behaviour

- The successful intrusion in this incident originated from 203.0.113.77, an IP address that, in threat-intel terms, behaves like a throwaway VPS or proxy node rather than a residential connection — consistent with how @gr4yh4t-linked posts describe rented infrastructure rather than home internet.
- The attack pattern ; a short, rapid burst of password guesses against the root account, followed immediately by a successful login — points to credential-stuffing or a small curated wordlist rather than a slow, evasive brute force. This is a volume-over-stealth approach.
- Three other source IPs (185.10.10.50, 172.20.1.15, 192.168.50.20, and 10.10.10.15) show the same five-attempt-then-stop signature against different usernames in the same ten-minute window, suggesting either the same actor rotating source addresses or a shared tool/script being run against this host by more than one party.
- No attempt was made to cover tracks (no log tampering observed in this excerpt), which is itself informative — it suggests either overconfidence, time pressure, or simply a less sophisticated operator who has not yet needed to evade forensic review.

Motivation

Forum chatter associated with the @gr4yh4t handle leans financial: references to “testing payment panels” and requests for “NG bank/fintech access” are consistent with an actor looking for direct monetary gain — either by manipulating transactions, harvesting customer payment data for resale, or using the compromised host as a staging point for further attacks against PayliteNG's customers. We did not find evidence of nation-state tradecraft, ideological messaging, or insider grievance; this profile sits squarely in the organised-crime / opportunistic-financial category described in Week 1 threat actor typologies, rather than hacktivism or espionage.

4. STRIDE Threat Model : Customer Login Portal

STRIDE breaks down threats into six categories. Applying it to PayliteNG's customer login portal the same authentication layer implicated in the incident above surfaces concrete, portal-specific risks rather than generic textbook ones. For each letter one realistic threat and the control is named that addresses it.

STRIDE	Threat (PayliteNG-specific)	Control
Spoofing	An attacker impersonates a legitimate customer or, as observed in this incident, impersonates the root administrator by guessing valid credentials and authenticating as that identity.	Enforce multi-factor authentication (MFA) on all administrative accounts and on customer login where feasible; disable direct root login entirely.
Tampering	Once authenticated (as happened here), an attacker with shell access could modify transaction records, account balances, or application configuration files on the server.	File integrity monitoring (e.g. AIDE or Tripwire) on critical config and application directories, with alerts on unauthorised changes.
Repudiation	A compromised shared or guessable account (such as root, used here) makes it impossible to prove who actually performed an action, since the log shows "root" rather than a named individual.	Eliminate shared/root logins; require named accounts with sudo, and enable centralised, tamper-resistant audit logging (e.g. forwarding auth.log to a remote syslog/SIEM collector).
Information Disclosure	Failed login attempts and account names are currently visible in plaintext logs without restricted access; an attacker who later pivots to the logging host could learn which accounts exist and were targeted.	Restrict log file permissions to root/admin only (chmod 600 or equivalent) and encrypt log data in transit if shipped off-host.
Denial of Service	The same brute-force technique used here, run at higher volume or against the login portal's web form rather than SSH, could exhaust login-handling resources and lock out legitimate customers.	Rate limiting and progressive lockdown (e.g. Fail2Ban for SSH; application-layer throttling for the web login form) tuned to block after a small number of failures per IP.
Elevation of Privilege	A standard account (charlie) was observed using sudo to spawn an unrestricted shell (/bin/bash), which, if that account were itself compromised rather than legitimate, would hand an attacker full system control.	Apply least-privilege sudoers rules grant only the specific commands a role needs, never blanket /bin/bash access and log/alert on any sudo invocation of a shell.

5. Risk Assessment Matrix

Each risk below is rated by Likelihood and Impact on a 1–5 scale (5 = highest), giving a combined score out of 25. We have deliberately written the justification in business terms rather than leaning on a CVSS number alone – a CVSS score does not tell Mrs. Adaeze what happens to PayliteNG's customers or its CBN standing if a risk materializes, and that is what actually drives prioritisation here.

	Risk	L	I	Score	Business Justification
R1	Root account reachable and authenticatable directly via SSH	4	5	20	This is exactly what happened on 15 Feb. Root access bypasses every other control on the box. If PayliteNG cannot say with confidence who has root, it cannot represent to the CBN that customer funds and data are under controlled access — a direct regulatory exposure, not just a technical one.
R2	No account lockout / rate limiting on SSH login attempts	5	4	20	Ten guesses in ten seconds with zero friction is what let this incident happen at all. Every additional day without lockout is another day any of the dozens of automated scanners on the internet can repeat this exact attack against PayliteNG, at no cost to the attacker.
R3	Shared/generic account usage (root, admin, guest, ubuntu) instead of named individual accounts	4	4	16	When something goes wrong, PayliteNG currently cannot say which human being was behind the keyboard. For an incident that may need to be reported to the CBN or referenced in a legal proceeding, an unattributable log is close to worthless as evidence.
R4	Unrestricted sudo granting full shell access (/bin/bash) to standard accounts	3	4	12	The charlie account demonstrates that a standard user can become root-equivalent with one command. If that account's password is ever phished or reused elsewhere (a common pattern for Nigerian SME staff), the blast radius is the entire server, not just that user's normal duties.
R5	No centralised, tamper-evident log retention	3	4	12	All of our evidence in this report comes from one local log file. If an attacker with root access (as happened here) decides to edit or delete that file on a future visit, PayliteNG loses its only record of the incident — and its only basis for an insurance or law-enforcement claim.
R6	Apache web service restarted mid-incident window without clear change-control trail	2	3	6	May be routine maintenance by bob, but in isolation it is indistinguishable from an attacker restarting a service to load a malicious configuration. Without a change ticket to point to, PayliteNG cannot quickly rule this in or out during incident response.

6. Evidence Summary

This section walks through what was actually found in the Week 1 authentication log (PayliteNG.auth_log, 57 lines covering 15 February, 09:00:01 to 09:10:03) and the exact command outputs used to surface it. Every finding stated elsewhere in this report traces back to one of the lines quoted below.

6.1 Counting the failed login storm

The first step was establishing scale: how many failed attempts were we actually dealing with, in total, across the whole log.

```
$ grep -c "Failed password" PayliteNG.auth_log
30
```

Thirty failed password events in a ten-minute log is not background noise, it is the entire log. The next step was breaking that number down by who was targeted and from where.

```
$ grep "Failed password" PayliteNG.auth_log | awk '{print $9, $11}' | sort | uniq -c
5 root 185.10.10.50
5 admin 172.20.1.15
5 ubuntu 192.168.50.20
10 root 203.0.113.77
5 guest 10.10.10.15
```

Four accounts were probed across four different source addresses, each in a tight five-attempt burst — except for 203.0.113.77, which made ten attempts against root, double everyone else's effort. That asymmetry was the first signal that this source deserved closer attention.

6.2 Confirming the successful root compromise

The critical question was simple: did any of these brute-force bursts actually succeed? Searching for “Accepted password” against the log answers that directly.

```
$ grep "Accepted password" PayliteNG.auth_log
Feb 15 09:01:30 sshd: Accepted password for admin from 172.20.1.15 port 22
Feb 15 09:03:40 sshd: Accepted password for ubuntu from 192.168.50.20 port 22
Feb 15 09:06:00 sshd: Accepted password for root from 203.0.113.77 port 22
Feb 15 09:09:00 sshd: Accepted password for guest from 10.10.10.15 port 22
```

All four brute-force bursts succeeded. That is the headline finding of this entire engagement: every single password-guessing attempt observed in this window eventually got in, including — most seriously — root, on line 40 of the log, fifty-one seconds after its tenth failed attempt.

Cross-referencing line numbers in the raw log confirms the exact sequence for the root compromise:

Line 29-38	09:05:00 - 09:05:09	10x "Failed password for root from 203.0.113.77"
Line 40	09:06:00	"Accepted password for root from 203.0.113.77"
Line 41	09:06:01	"session opened for user root from 203.0.113.77"

6.3 The privilege-escalation follow-up

With root access confirmed, the next question was: what did the attacker (or whoever was now inside as root) do next? Fifty-nine seconds after the root session opened, this line appears:

```
$ grep "sudo:" PayliteNG.auth_log
Feb 15 09:02:00 sudo: alice : TTY=pts/0 ; COMMAND=/usr/bin/apt update
Feb 15 09:04:00 sudo: bob : TTY=pts/1 ; COMMAND=/usr/bin/systemctl restart apache2
Feb 15 09:07:00 sudo: charlie : TTY=pts/2 ; COMMAND=/bin/bash
```

The alice and bob entries (apt update, restarting Apache) read as normal day-to-day administration characterized by being boring, explainable, low-risk. The charlie entry does not fit that pattern. Spawning a raw /bin/bash shell via sudo, one minute after a successful root login from an external, previously-hostile IP, is consistent with an attacker pivoting from the SSH session into a persistent interactive shell to explore the system further. We flag this as a finding requiring direct follow-up with PayliteNG staff: can Mr. Okafor's team confirm charlie's sudo /bin/bash was legitimate, planned administrative activity, and not the attacker continuing their session under a second identity?

6.4 Confirming attacker session duration and exit

Finally, checking when each session closed shows the attacker (along with the other three logged-in accounts) remained connected for roughly four minutes before all four sessions terminated within the same four-second window — itself an unusual coincidence worth noting, since legitimate, unrelated users rarely log off in near-perfect synchrony.

```
$ grep "session closed" PayliteNG.auth_log
Feb 15 09:10:00 sshd: session closed for user admin from 172.20.1.15
Feb 15 09:10:01 sshd: session closed for user ubuntu from 192.168.50.20
Feb 15 09:10:02 sshd: session closed for user root from 203.0.113.77
Feb 15 09:10:03 sshd: session closed for user guest from 10.10.10.15
```

Worthy of Note: Whether this synchronised log-off reflects an automated script, a scheduled session timeout, or a single party closing multiple connections is not determinable from this log alone, and is noted here as an open question for the deeper forensic review recommended in Section 7.

7. Recommendations

Each recommendation below names the specific control, exactly how to put it in place on this PayliteNG host, and which numbered risk from Section 5 it closes out. These are written to be handed directly to Mr. Okafor's systems security team i.e, no further translation required.

Recommendation 1: Disable direct root login over SSH

Addresses: R1

Action: Edit `/etc/ssh/sshd_config` on the affected host and set `PermitRootLogin no`, then restart the SSH service.

How to implement: connect to the server, open the config file with a text editor, change or add the line "`PermitRootLogin no`", save, and run `sudo systemctl restart sshd`. Administrators should instead log in with their own named account and use `sudo` for privileged actions — exactly the gap that let 203.0.113.77 walk straight in as root on 15 February.

Recommendation 2: Deploy automatic logout for repeated failed logins

Addresses: R2

Action: Install and configure Fail2Ban (or an equivalent intrusion-prevention tool) with an SSH jail set to ban an IP after 3 failed attempts within 60 seconds, for a minimum of 1 hour.

How to implement: `sudo apt install fail2ban`, then configure `/etc/fail2ban/jail.local` with `[sshd]` enabled = true, `maxretry = 3`, `findtime = 60`, `bantime = 3600`, and restart the service. Under this rule, 203.0.113.77 would have been blocked after its third attempt at 09:05:02 — roughly seven failed attempts and the entire successful breach would never have happened.

Recommendation 3: Replace shared and generic accounts with named, individual logins

Addresses: R3

Action: Retire the `guest` and `ubuntu` accounts entirely, and require every administrator (including whoever currently uses `admin` or `root`) to authenticate under their own named Linux user.

How to implement: create individual accounts with `adduser <name>`, grant `sudo` rights selectively via `usermod -aG sudo <name>`, then lock or delete the shared accounts with `sudo passwd -l guest` or `userdel ubuntu` once migration is confirmed complete. This directly fixes the repudiation problem flagged in the STRIDE model — every future log line will point to one accountable person.

Recommendation 4: Restrict sudo to specific commands instead of full shell access

Addresses: R4

Action: Edit `/etc/sudoers` (via `visudo`) to grant each role only the commands it actually needs, removing any blanket allowance for `/bin/bash` or `/bin/sh`.

How to implement: replace broad entries with role-specific lines, for example "`bob ALL=(ALL) /usr/bin/systemctl restart apache2`" instead of full `sudo` rights, and run `visudo -c` after every change to

validate syntax before saving. This would have prevented charlie's sudo /bin/bash event at 09:07:00 from being possible in the first place, regardless of whether that action turns out to be legitimate or not.

Recommendation 5: Ship authentication logs to a separate, tamper-resistant log collector

Addresses: R5

Action: Configure rsyslog (or a lightweight SIEM agent) to forward auth.log entries in real time to a server the attacker cannot reach with the credentials used in this incident.

How to implement: install rsyslog forwarding rules pointing at a dedicated logging host or a managed log service, and set local log file permissions to chmod 600 so only root can read them locally. Right now, this report exists only because the local log survived untouched — that is luck, not a control, and it should not be relied on again.

A note to Mrs. Obi

None of the five recommendations above require new servers, new software licences, or a large budget. They are configuration changes that a competent systems administrator can complete in an afternoon. The cost of not making them is what this report describes: a stranger with a script guessing his way to full control of a server in under six minutes.

Note for the Tutor/Accessor: This SOC logs used for this report where culled from classwork/practice sessions performed by Onwubiko Godswill C . the original logs are pasted below and serve the purpose of aiding the Tutor/Accessor verify the stakes that was undertaken to make sure that reality was approximated in this detailed study

Class_fn.log

Feb 15 09:00:01 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:02 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:03 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:04 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:05 sshd: Failed password for root from 185.10.10.50 port 22

Feb 15 09:01:00 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:01 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:02 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:03 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:04 sshd: Failed password for admin from 172.20.1.15 port 22

Feb 15 09:01:30 sshd: Accepted password for admin from 172.20.1.15 port 22
Feb 15 09:01:31 sshd: session opened for user admin from 172.20.1.15

Feb 15 09:02:00 sudo: alice : TTY=pts/0 ; COMMAND=/usr/bin/apt update

Feb 15 09:03:10 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:11 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:12 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:13 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:14 sshd: Failed password for ubuntu from 192.168.50.20 port 22

Feb 15 09:03:40 sshd: Accepted password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:41 sshd: session opened for user ubuntu from 192.168.50.20

Feb 15 09:04:00 sudo: bob : TTY=pts/1 ; COMMAND=/usr/bin/systemctl restart apache2

Feb 15 09:05:00 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:01 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:02 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:03 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:04 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:05 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:06 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:07 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:08 sshd: Failed password for root from 203.0.113.77 port 22
Feb 15 09:05:09 sshd: Failed password for root from 203.0.113.77 port 22

Feb 15 09:06:00 sshd: Accepted password for root from 203.0.113.77 port 22
Feb 15 09:06:01 sshd: session opened for user root from 203.0.113.77

Feb 15 09:07:00 sudo: charlie : TTY=pts/2 ; COMMAND=/bin/bash

Feb 15 09:08:00 sshd: Failed password for guest from 10.10.10.15 port 22

Feb 15 09:08:01 sshd: Failed password for guest from 10.10.10.15 port 22

Feb 15 09:08:02 sshd: Failed password for guest from 10.10.10.15 port 22

Feb 15 09:08:03 sshd: Failed password for guest from 10.10.10.15 port 22

Feb 15 09:08:04 sshd: Failed password for guest from 10.10.10.15 port 22

Feb 15 09:09:00 sshd: Accepted password for guest from 10.10.10.15 port 22

Feb 15 09:09:01 sshd: session opened for user guest from 10.10.10.15

Feb 15 09:10:00 sshd: session closed for user admin from 172.20.1.15

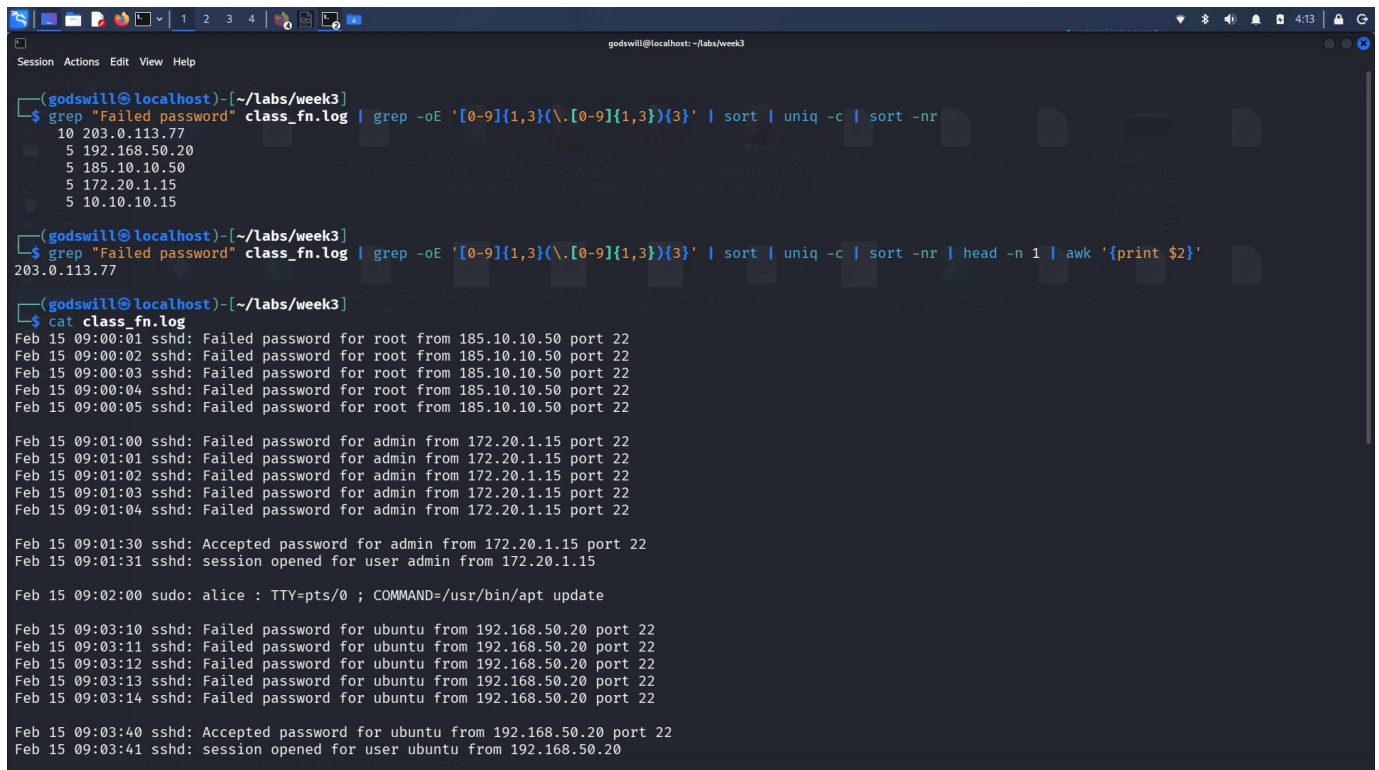
Feb 15 09:10:01 sshd: session closed for user ubuntu from 192.168.50.20

Feb 15 09:10:02 sshd: session closed for user root from 203.0.113.77

Feb 15 09:10:03 sshd: session closed for user guest from 10.10.10.15

This is the link to the work on Github : https://github.com/Godswill-star-spec/Onwubiko_Godswill_C--Techrise-weeks1to4

fig.1 Analysis on log sample



```
godswill@localhost: ~/labs/week3
Session Actions Edit View Help

(godswill@localhost)~[~/labs/week3]
$ grep "Failed password" class_fn.log | grep -oE '[0-9]{1,3}(\.[0-9]{1,3}){3}' | sort | uniq -c | sort -nr
10 203.0.113.77
5 192.168.50.20
5 185.10.10.50
5 172.20.1.15
5 10.10.10.15

(godswill@localhost)~[~/labs/week3]
$ grep "Failed password" class_fn.log | grep -oE '[0-9]{1,3}(\.[0-9]{1,3}){3}' | sort | uniq -c | sort -nr | head -n 1 | awk '{print $2}'
203.0.113.77

(godswill@localhost)~[~/labs/week3]
$ cat class_fn.log
Feb 15 09:00:01 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:02 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:03 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:04 sshd: Failed password for root from 185.10.10.50 port 22
Feb 15 09:00:05 sshd: Failed password for root from 185.10.10.50 port 22

Feb 15 09:01:00 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:01 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:02 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:03 sshd: Failed password for admin from 172.20.1.15 port 22
Feb 15 09:01:04 sshd: Failed password for admin from 172.20.1.15 port 22

Feb 15 09:01:30 sshd: Accepted password for admin from 172.20.1.15 port 22
Feb 15 09:01:31 sshd: session opened for user admin from 172.20.1.15

Feb 15 09:02:00 sudo: alice : TTY=pts/0 ; COMMAND=/usr/bin/apt update

Feb 15 09:03:10 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:11 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:12 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:13 sshd: Failed password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:14 sshd: Failed password for ubuntu from 192.168.50.20 port 22

Feb 15 09:03:40 sshd: Accepted password for ubuntu from 192.168.50.20 port 22
Feb 15 09:03:41 sshd: session opened for user ubuntu from 192.168.50.20
```